



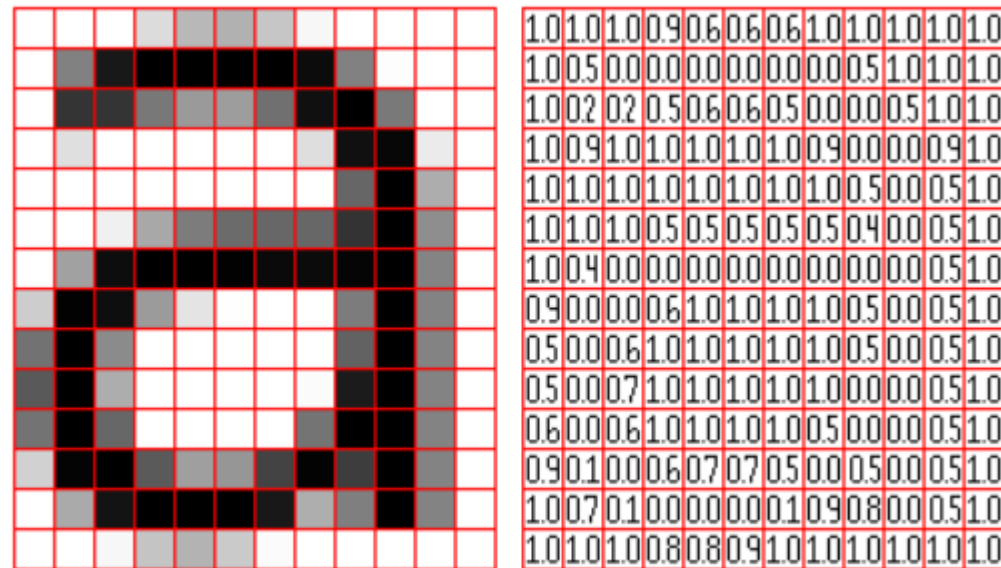
Convolution Neural Network – Better Understanding!

Introduction

- ❑ Images make up a special class of input data.
- ❑ We use pictures and photographs to communicate all kinds of things for professional, social, and personal reasons.
- ❑ We want to label the face of a loved one so we can easily find them in photographs
- ❑ Determine whether a stick-figure drawing is a person or a cat
- ❑ Judge whether a smudge on a radiograph is a medical condition requiring a closer look
- ❑ **extracting meaning from images is important.**

Introduction

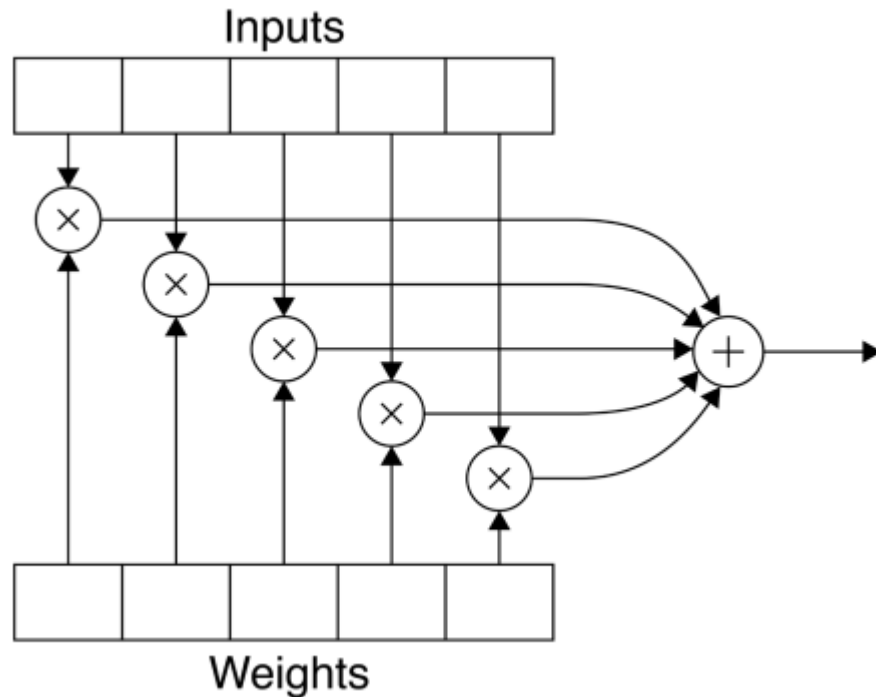
- ❑ The eye and the visual brain are extremely complex and hierarchical.
- ❑ So the same process, is that possible for computers? and the simple answer is YES!.
- ❑ Computers can SEE the world but in a different way – through numbers.



Introduction

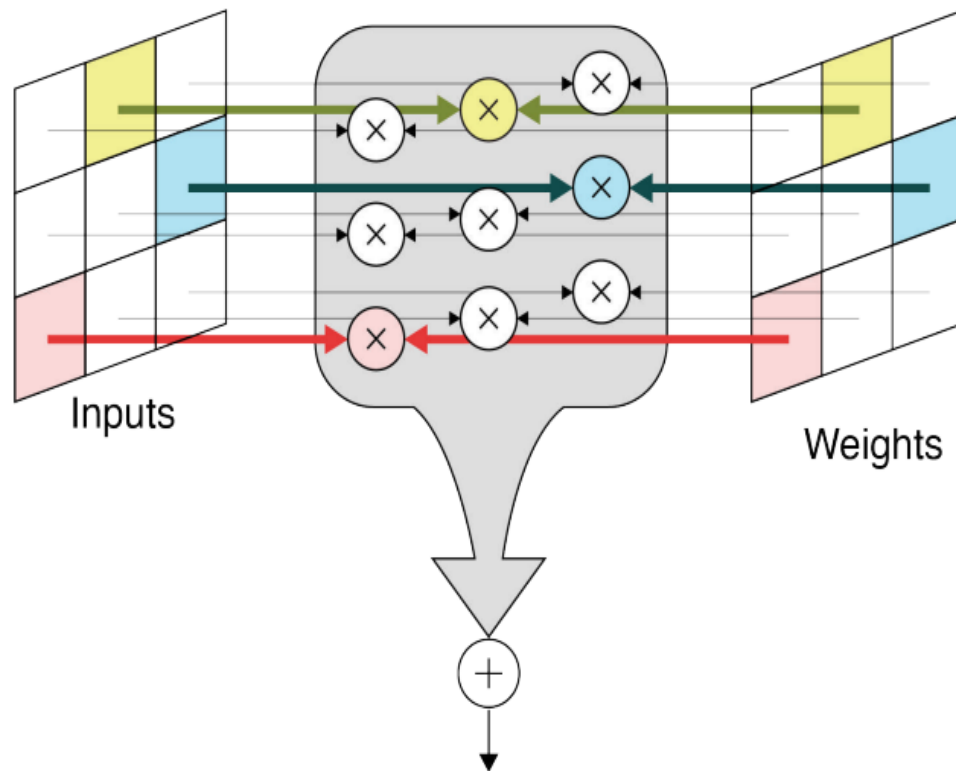
- ❑ The importance of layers of neurons arranged in each algorithm is, each layer concentrates each pattern and its importance like firstly it observes simple patterns such as lines, curves,
- ❑ Then some complex patterns like texts, objects, and face and
- ❑ more complex patterns present in the image

Convolution in 1-D



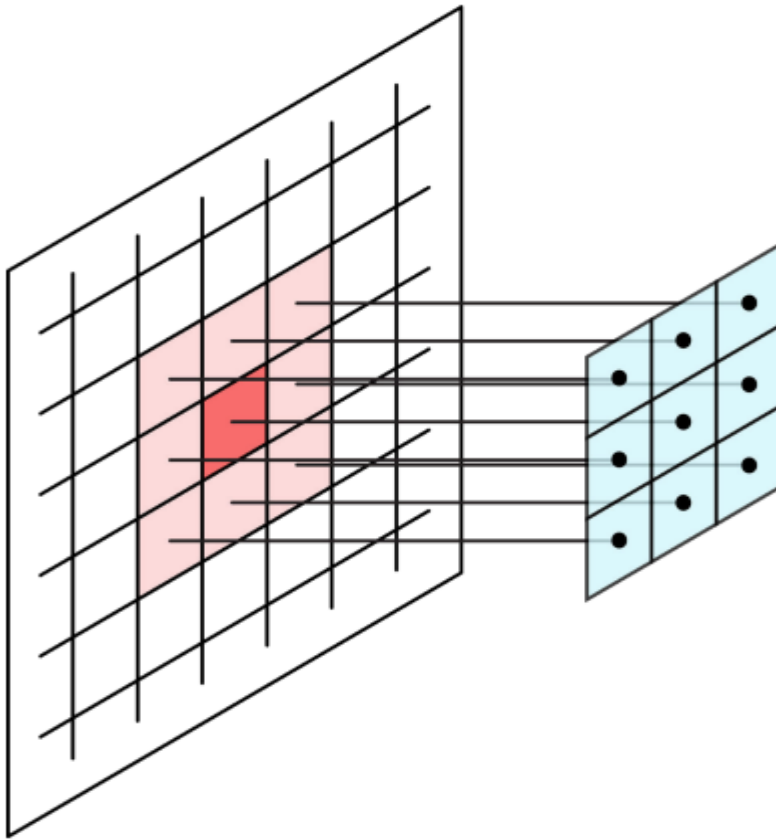
A step of convolution involves two lists, which we can call “inputs” and “weights,” though both lists are treated the same way. Corresponding values are multiplied together pairwise, and then the results are added together. This is what an artificial neuron does, ignoring the activation function.

Convolution in 2-D



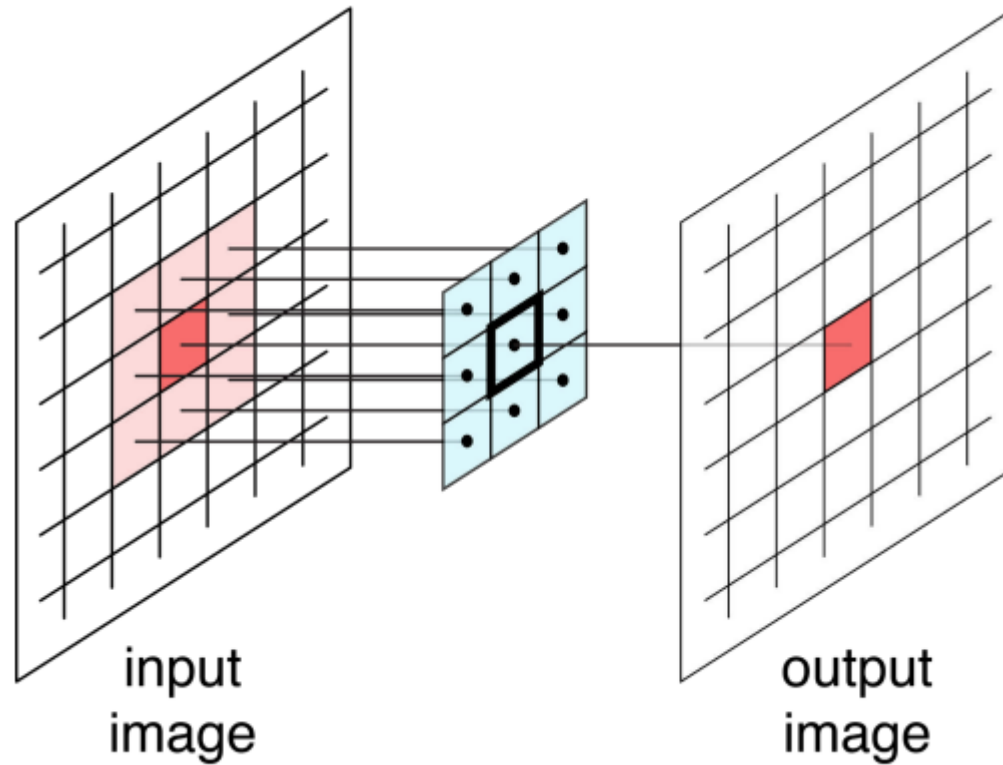
Convolution in 2D works just like in 1D, though the picture gets more cluttered. Each pair of corresponding values gets multiplied together, and then all of those products are added together

Convolution in 2-D



The neuron gets inputs in this case from a 3 by 3 square around the pixel. The location of the pixel in the image is highlighted in bright red. The set of all 9 pixels is called the local receptive field for the neuron. Our input image here has one channel, so the neuron receives 9 inputs. Each is multiplied by a corresponding weight, shown in blue.

Convolution in 2-D

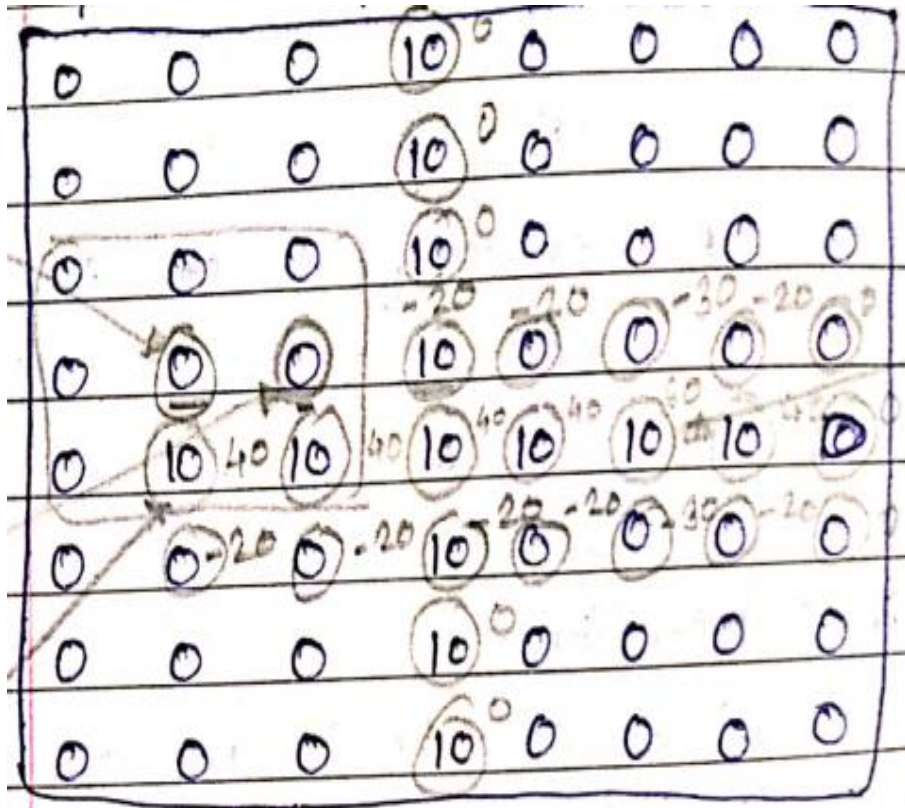


The neuron has a 3 by 3 receptive field, with the highlighted anchor at the center.

The bright red pixel in the input image, called the focus pixel, shows where the anchor is currently located.

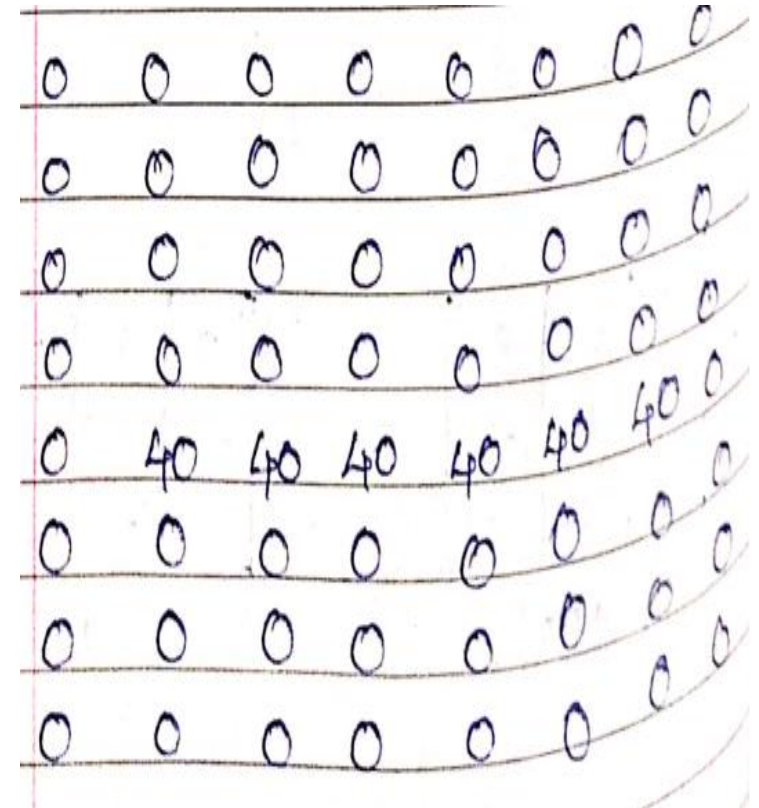
The output of the neuron goes into the output image at the same location as the focus pixel.

Convolution in 2-D

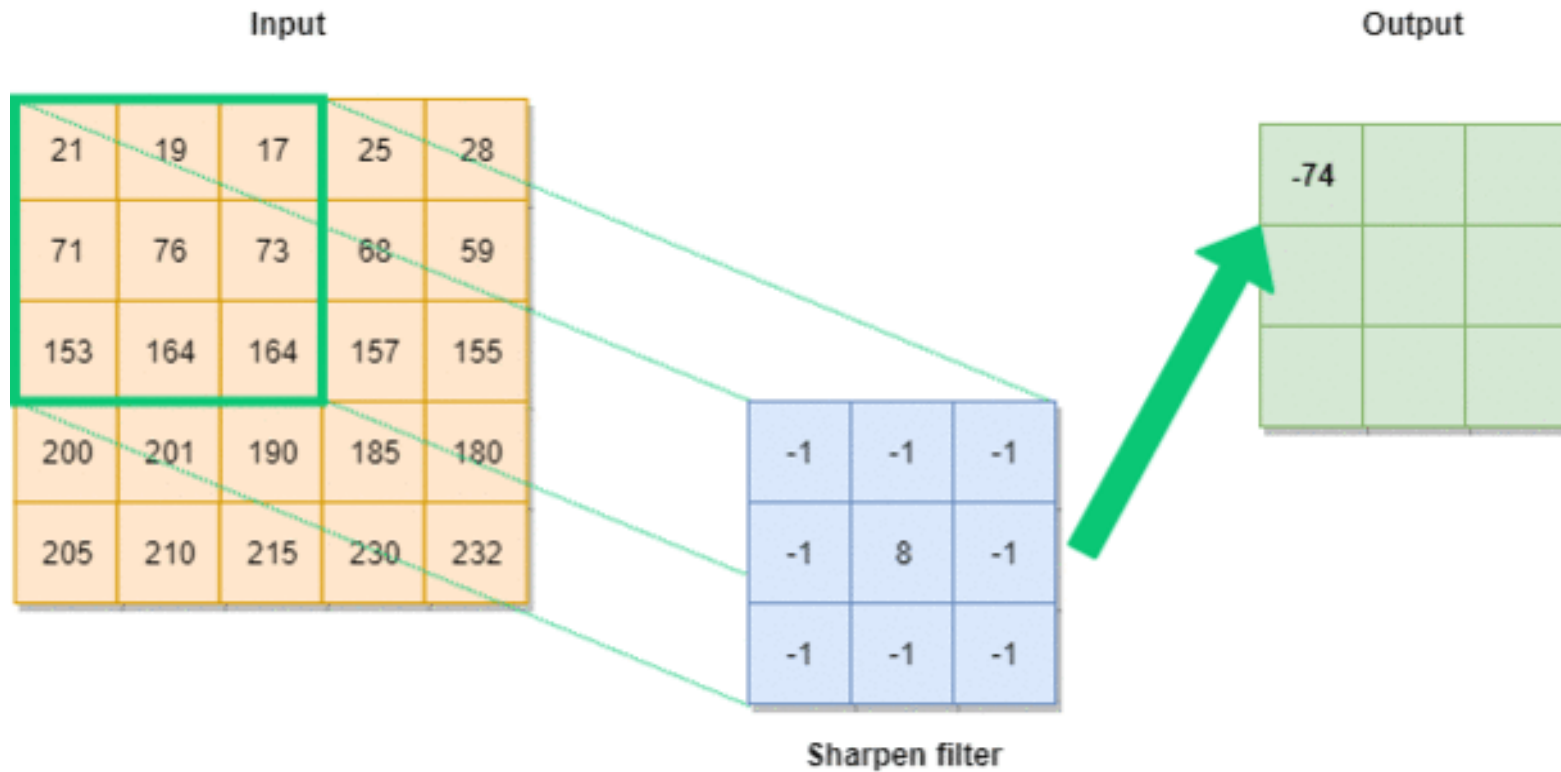


-1	-1	-1
2	2	2
-1	-1	-1

Horizontal

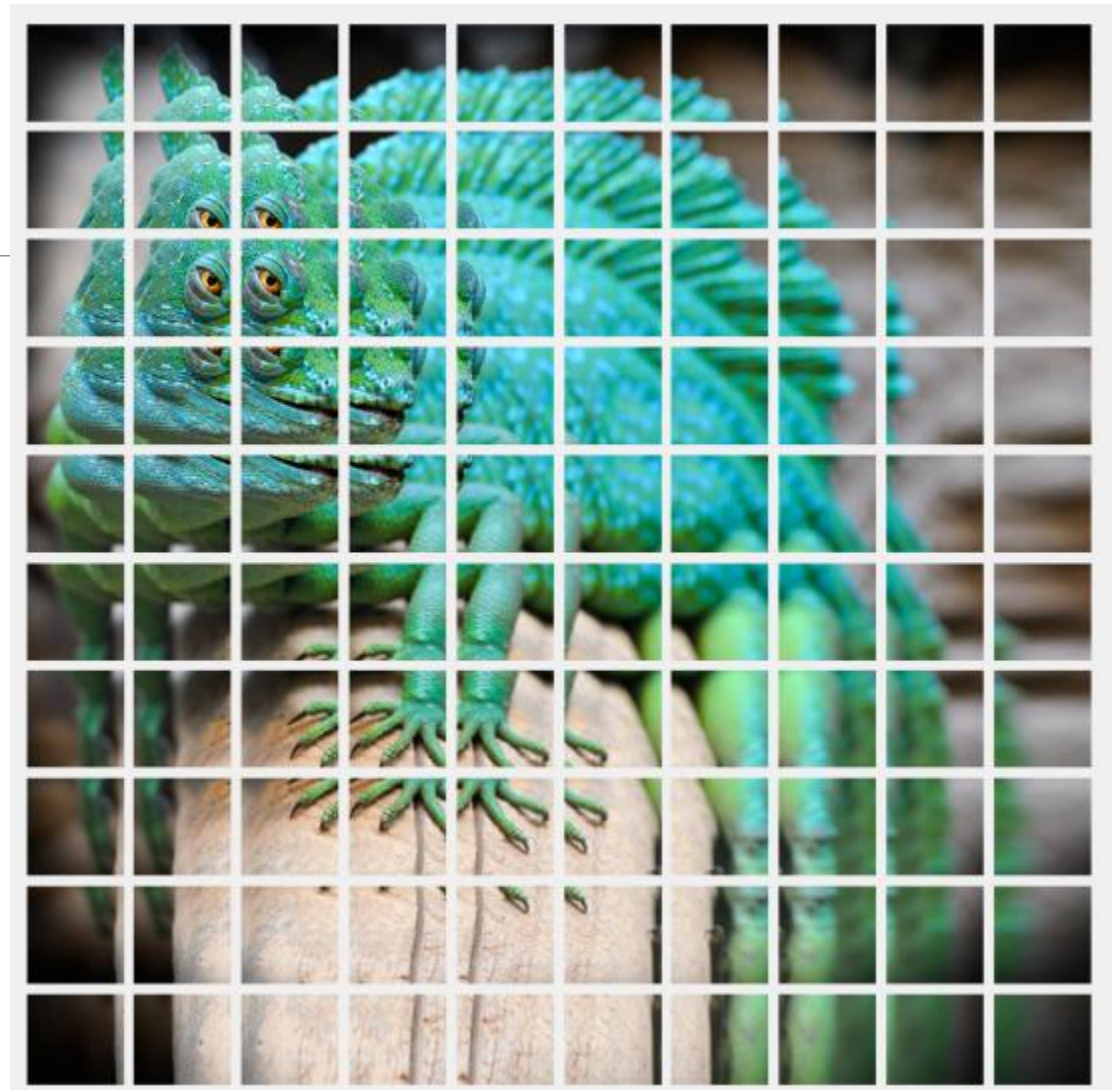


Convolution in 2-D



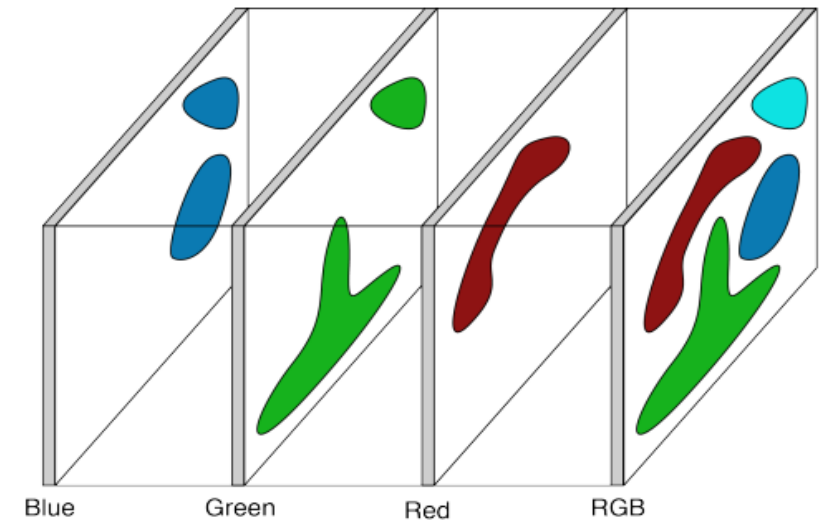
AIGeekProgrammer.com © 2019

Fly's Eye



Depth

- ❑ In NN, each image is a sample
- ❑ Each pixel in a grayscale image is a single feature.
- ❑ If the image is in color, then there are three features per sample (one each for the red, green, and blue the values at each pixel).



- ❑ A grayscale image represents its pixel data with 1 channel
- ❑ A color image required 3 channels (one each for red, green, and blue) to represent pixel data

Depth

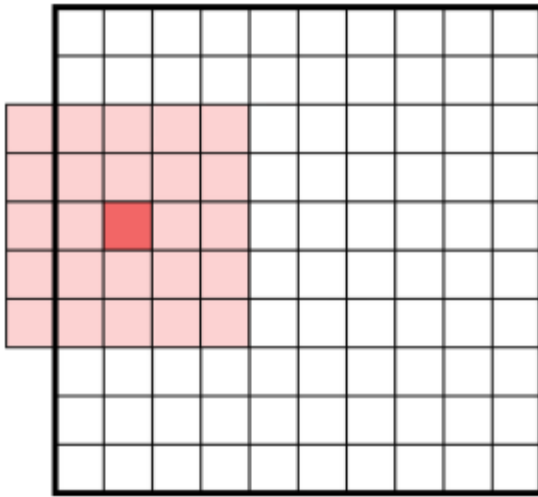
- ❑ Thus, Every image has a size, given by its width and height.
- ❑ It will also have **depth = the number of channels.**
- ❑ Thus we say that a grayscale image has a depth of 1, while a color image (with one channel each for red, green, and blue) has a depth of 3.

The idea is that we add a border of “extra” pixels around the outside of the image.

Padding

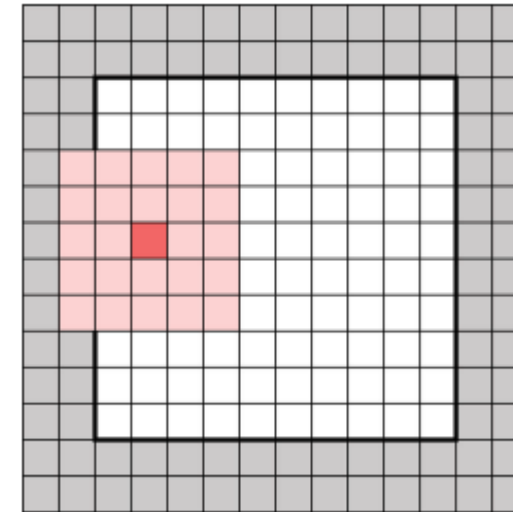
By far the most common choice is simply zero.

This choice is called zero-padding.



Near the edge, the filter's receptive field can fall off the side of the image.

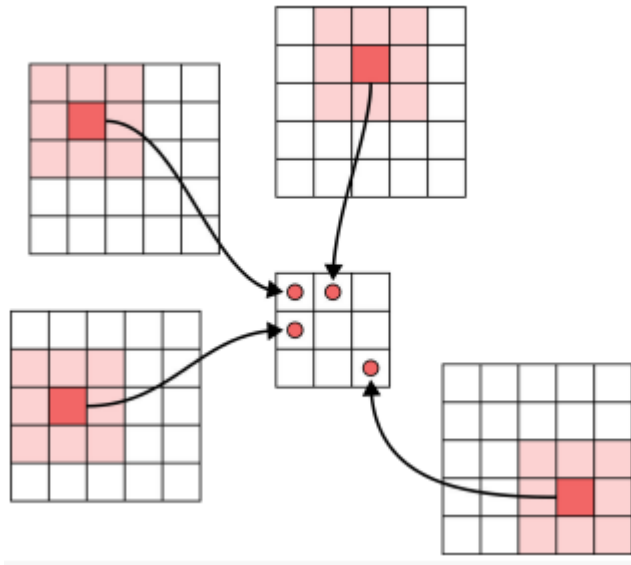
What values do we use for these missing pixels?



A better way to solve the “falling off the edge” problem is to add padding, or extra pixels, around the border of the image.

Here we've added a 2-pixel border, so every pixel in the original image (shown in white) can be used as the center of the filter. Usually, the padded pixels are given the value 0.

Stride



Sweeping a 3 by 3 filter over a 5 by 5 image, without padding. Each step of the filter moves it to the right by one pixel in the input, and then it moves down by one pixel. The diagram shows 4 of the 9 filter positions that would make up this output.

Stride

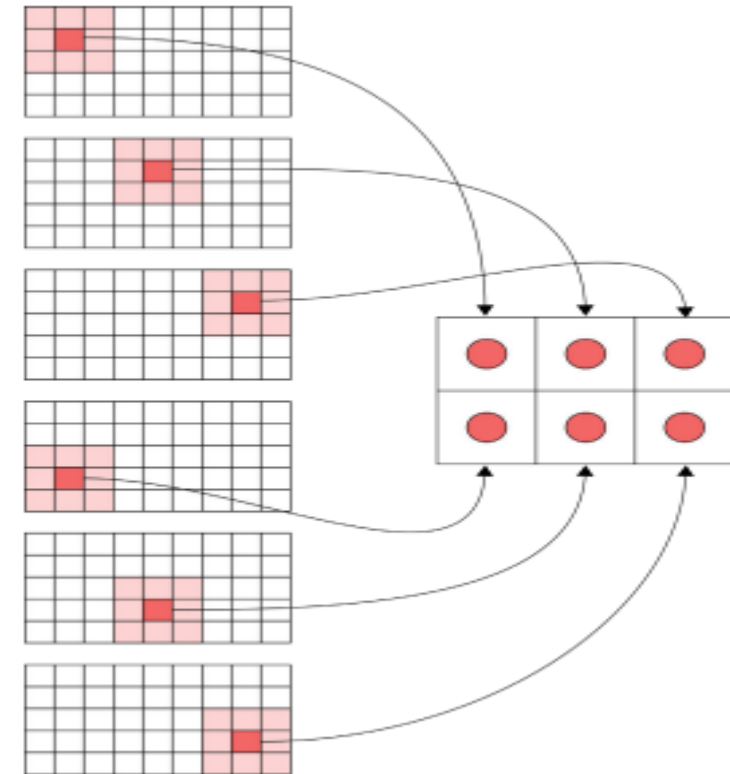
Our input scanning can skip over pixels.

In this example, we look only at every third pixel horizontally, and every other pixel vertically.

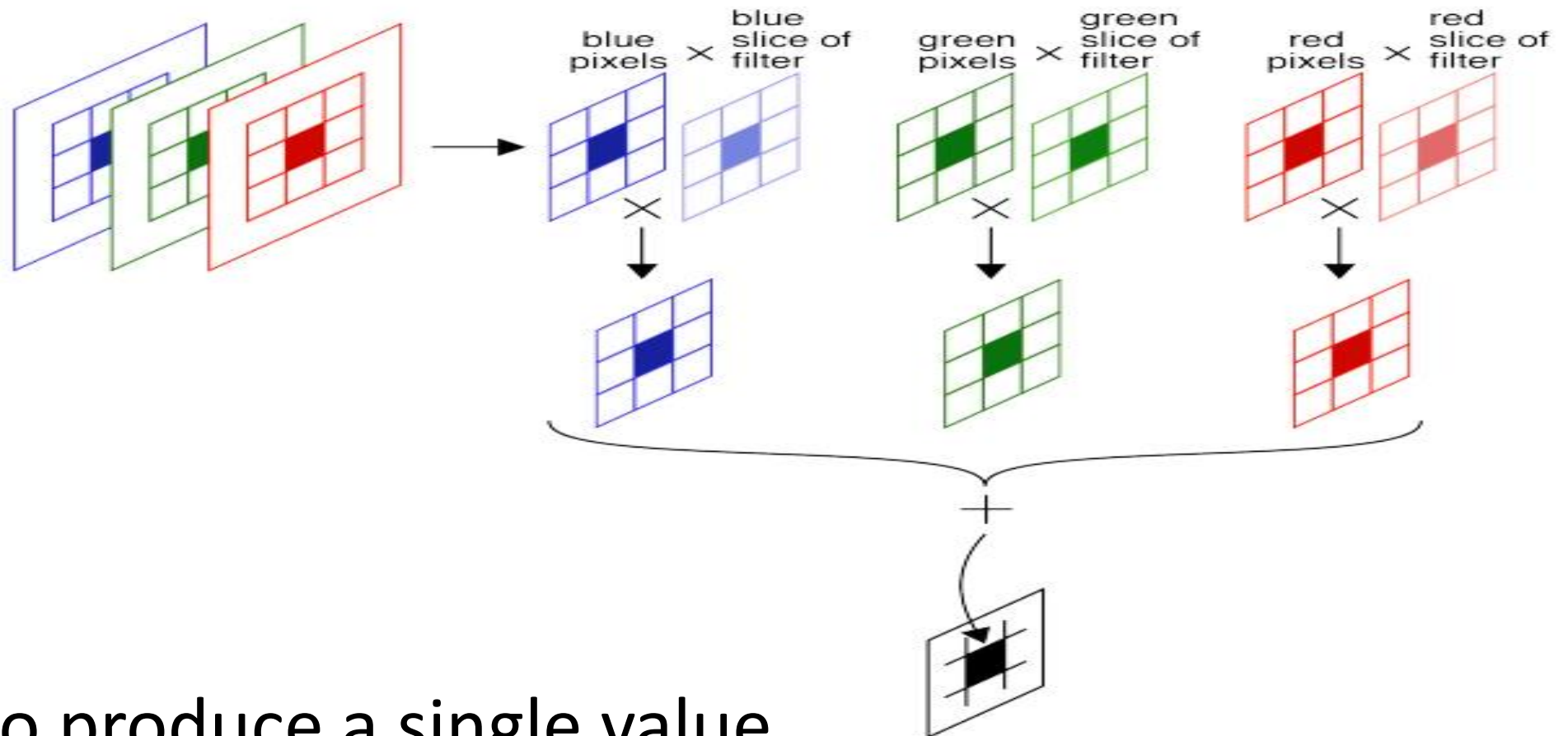
In other words, we use a stride of 3 horizontally and 2 vertically. For clarity, we're not using padding in this figure.

The 5 by 9 input image turns into a 2 by 3 output.

The number of rows and columns traversed per slide are referred as - Stride

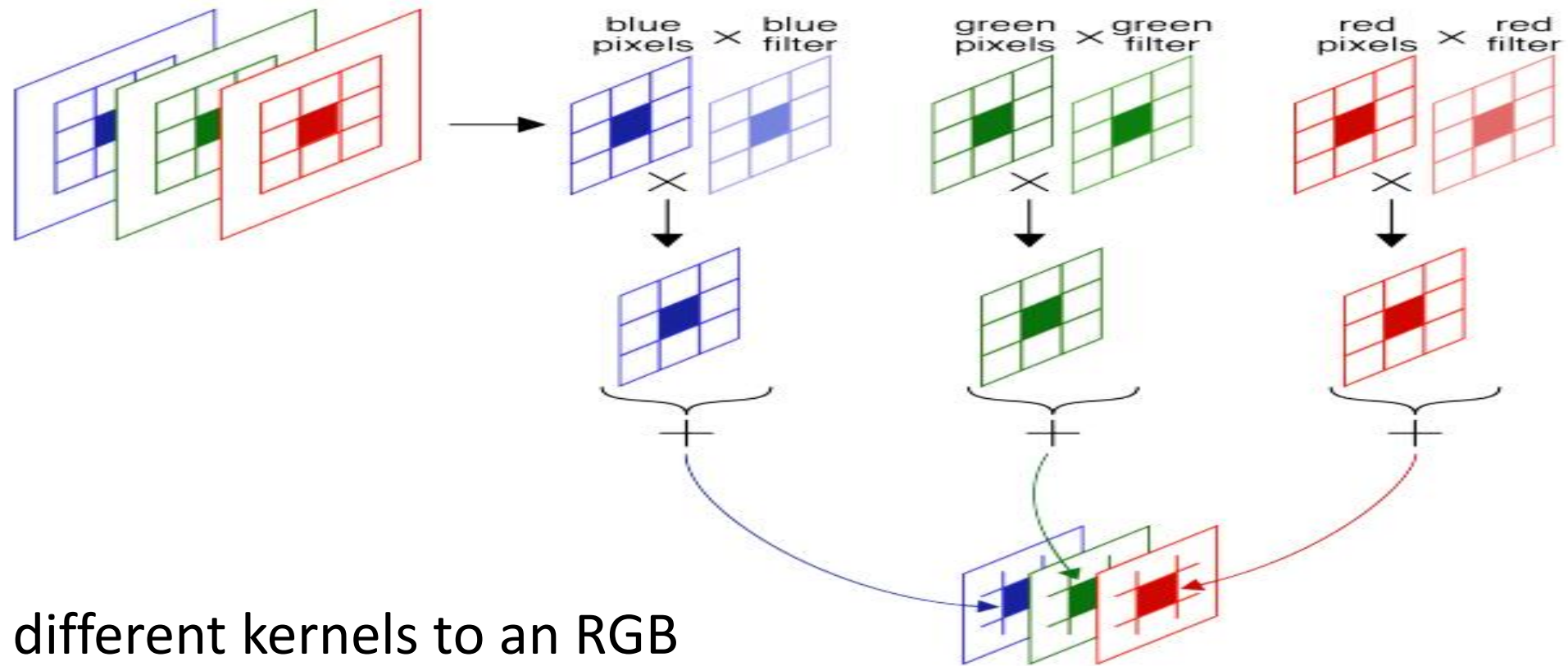


High-Dimensional Convolution



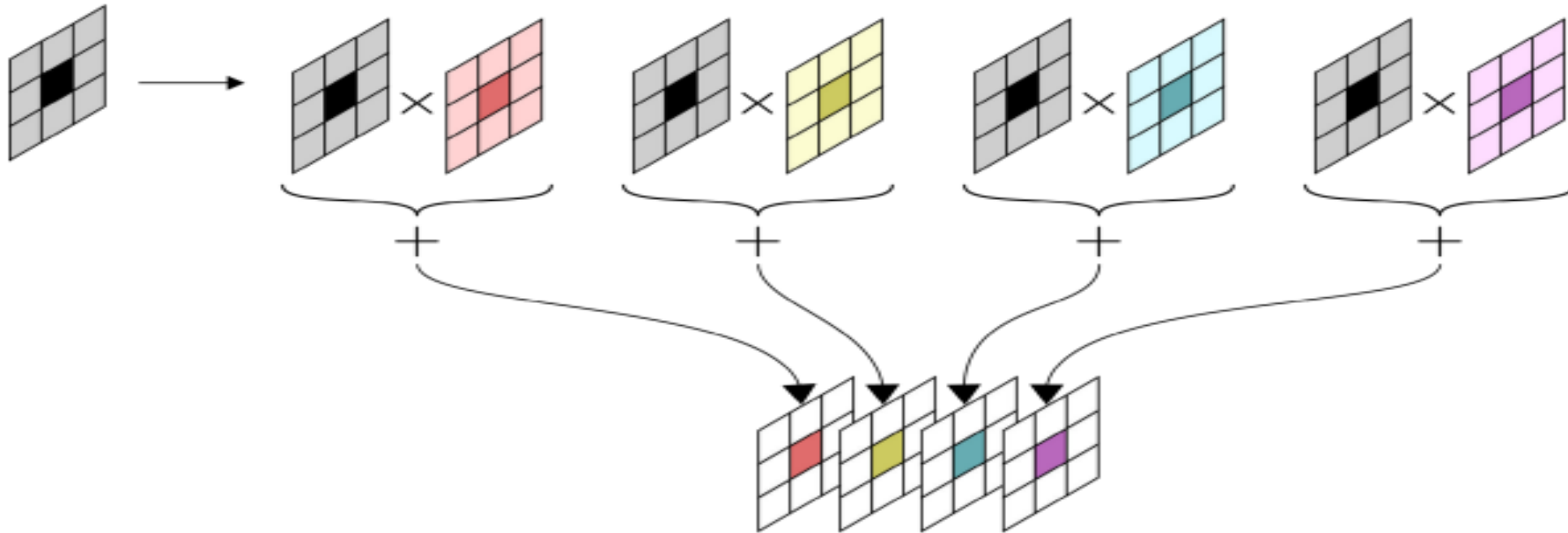
to produce a single value

High-Dimensional Convolution



apply 3 different kernels to an RGB image to create an output RGB image.

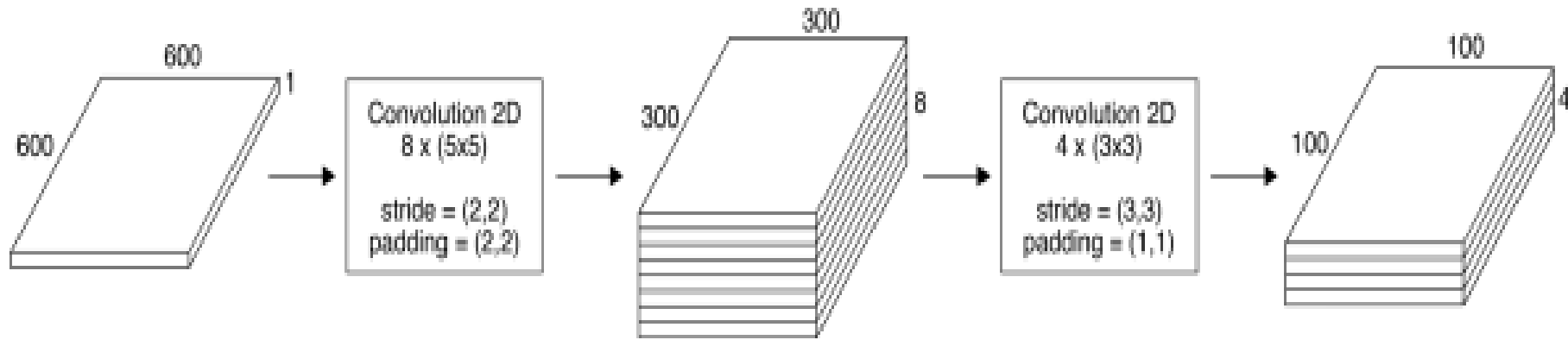
Filters with Multiple Channels



We can run multiple filters (in color) over the same input (in gray). Each filter creates its own channel in the output.

Striding for Hierarchies

Each stage uses a lower-resolution version of the previous stage,
so it can work with **larger collections of features** without requiring larger filters.



Creating a hierarchy of convolutions.

The first convolution works on the full 600 by 600 image.

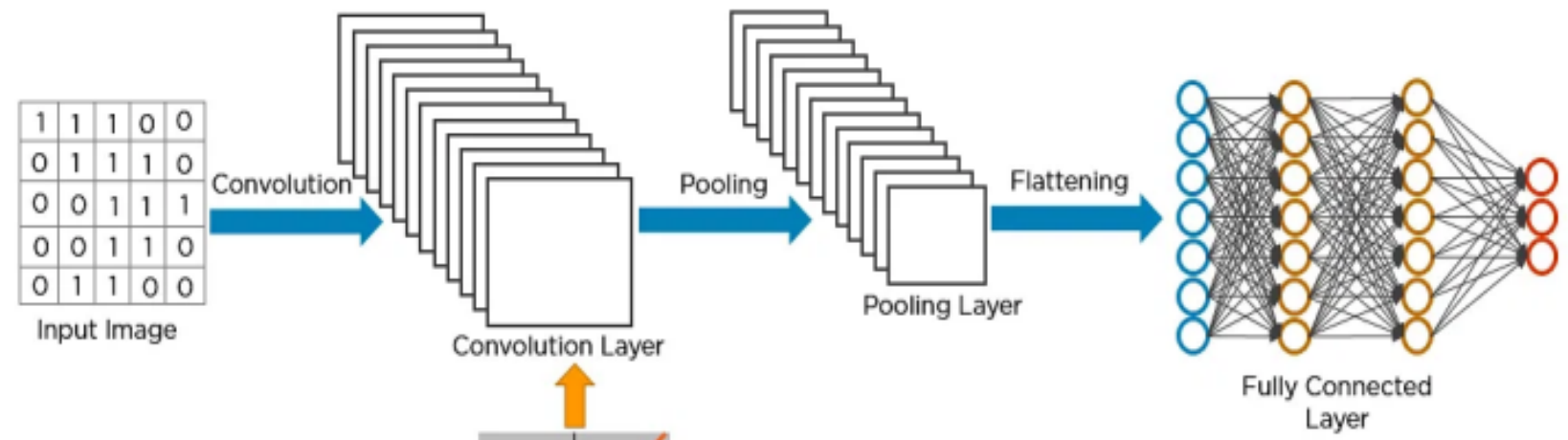
The second works on a 300 by 300 version of that image that contains the results of the first stage's 5 filters.

Input, Output and Filter Size

- ❑ Number of parameter in the convolution layer = $\text{kernel_size} + \text{bias}$
- ❑ Output shape = $(n - f + 2p) / s + 1$
 - ❑ where s – stride, p -padding, n - input size and f - filter size
- ❑ Given filters =1, kernel size = (3,3) and input shape = (10,10,1)
- ❑ Calculate number of parameters and output shape

CNN Architecture

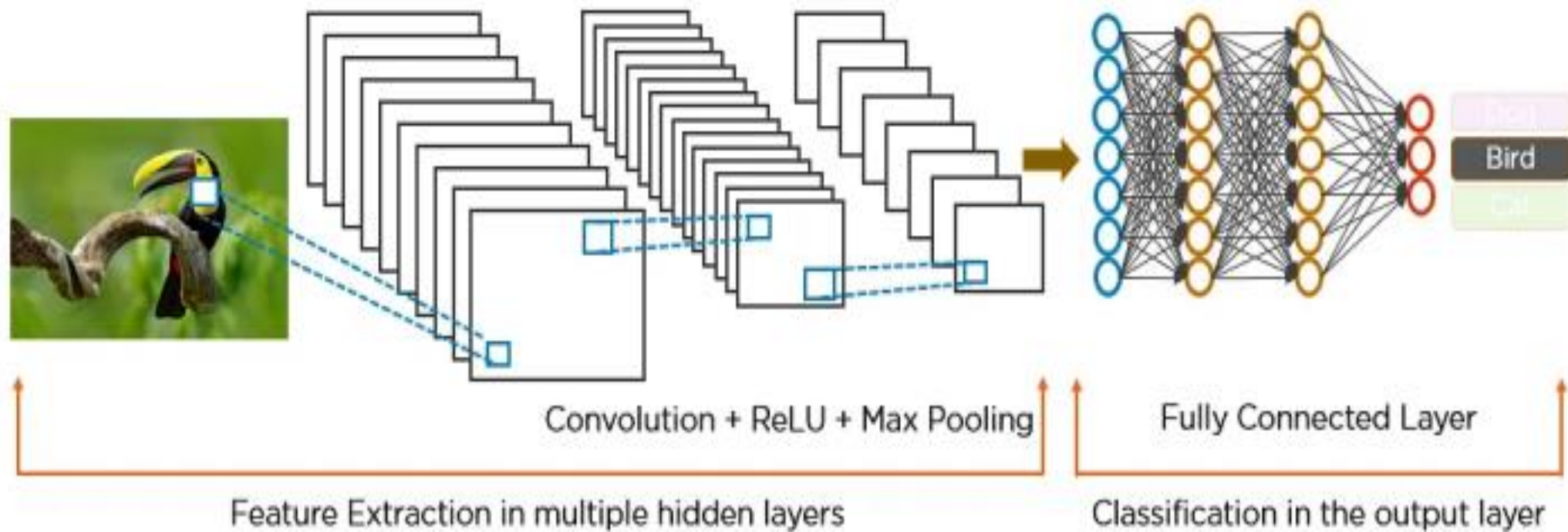
- ❑ A convolution neural network has multiple hidden layers that help in extracting information from an image.
- ❑ The three important layers in CNN are:
 - ❑ 1. Convolution layer
 - ❑ 2. Pooling layer
 - ❑ 3. Fully connected layer



Classification Process

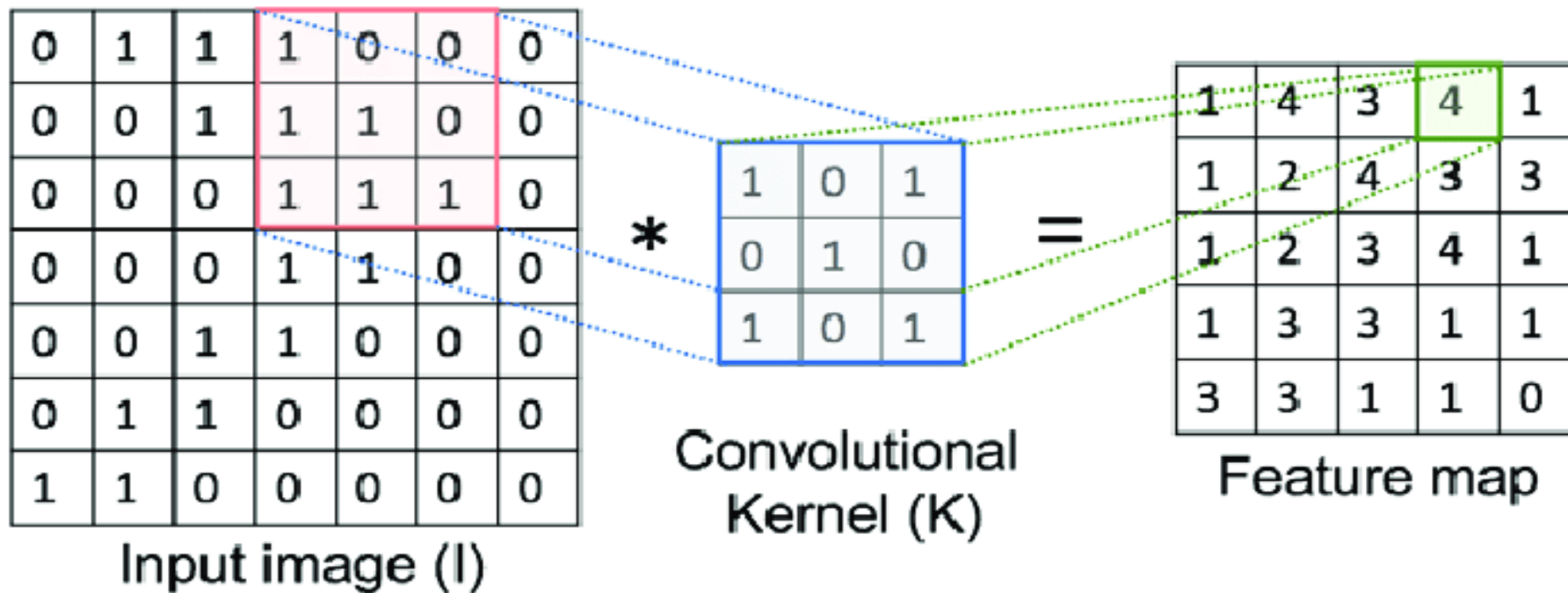
- ❑ The pixels from the image are fed to the **convolutional layer** that performs the convolution operation
- ❑ It results in a convolved map
- ❑ The convolved map is applied to a **ReLU function** to generate a rectified feature map
- ❑ The image is processed with multiple convolutions and ReLU layers for locating the features
- ❑ Different **pooling layers** with various filters are used to identify specific parts of the image
- ❑ The pooled feature map is flattened and fed to a **fully connected layer** to get the final output

CNN Example – to classify as a bird



Convolution Layer

- This is the first step in the process of extracting valuable features from an image.
- A convolution layer has several filters that perform the convolution operation.



Convolution Layer – reduction in size

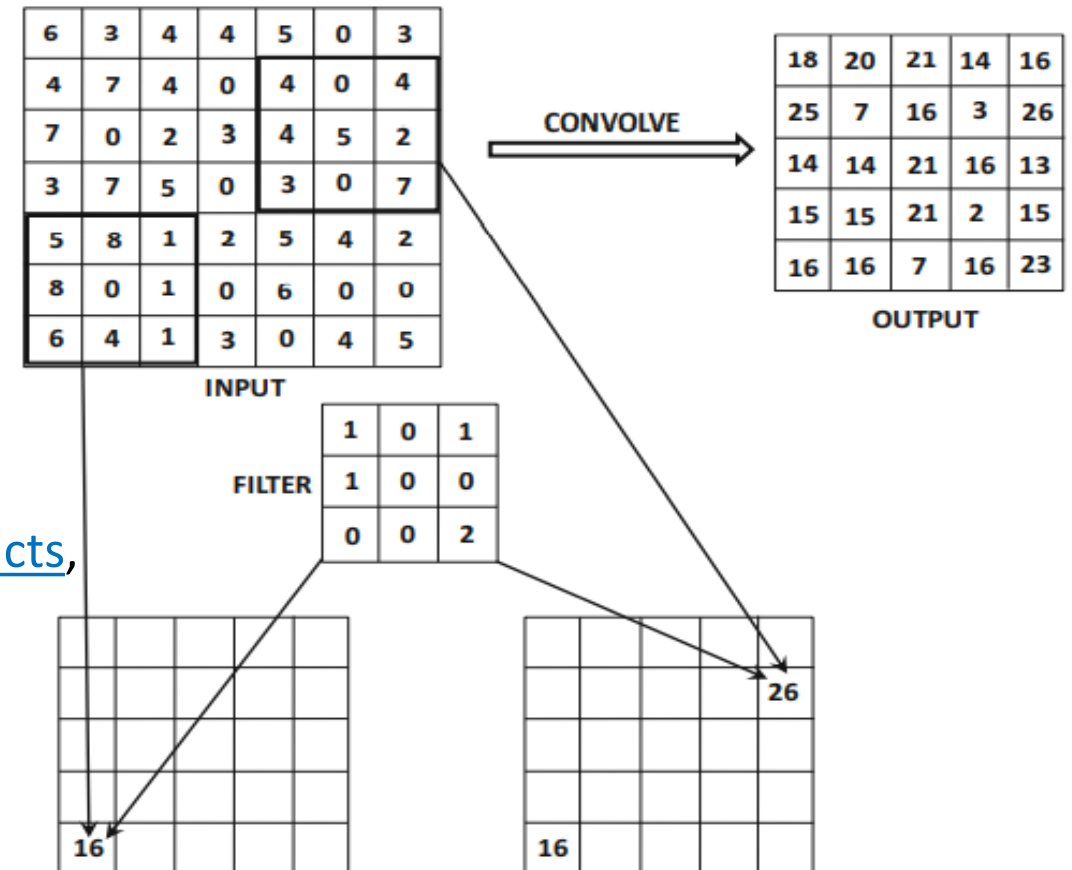
When performing convolutions in the q th layer, one can align the filter at (L_{q+1}, B_{q+1})

$L_{q+1} = (L_q - F_q + 1)$ positions along the height
and

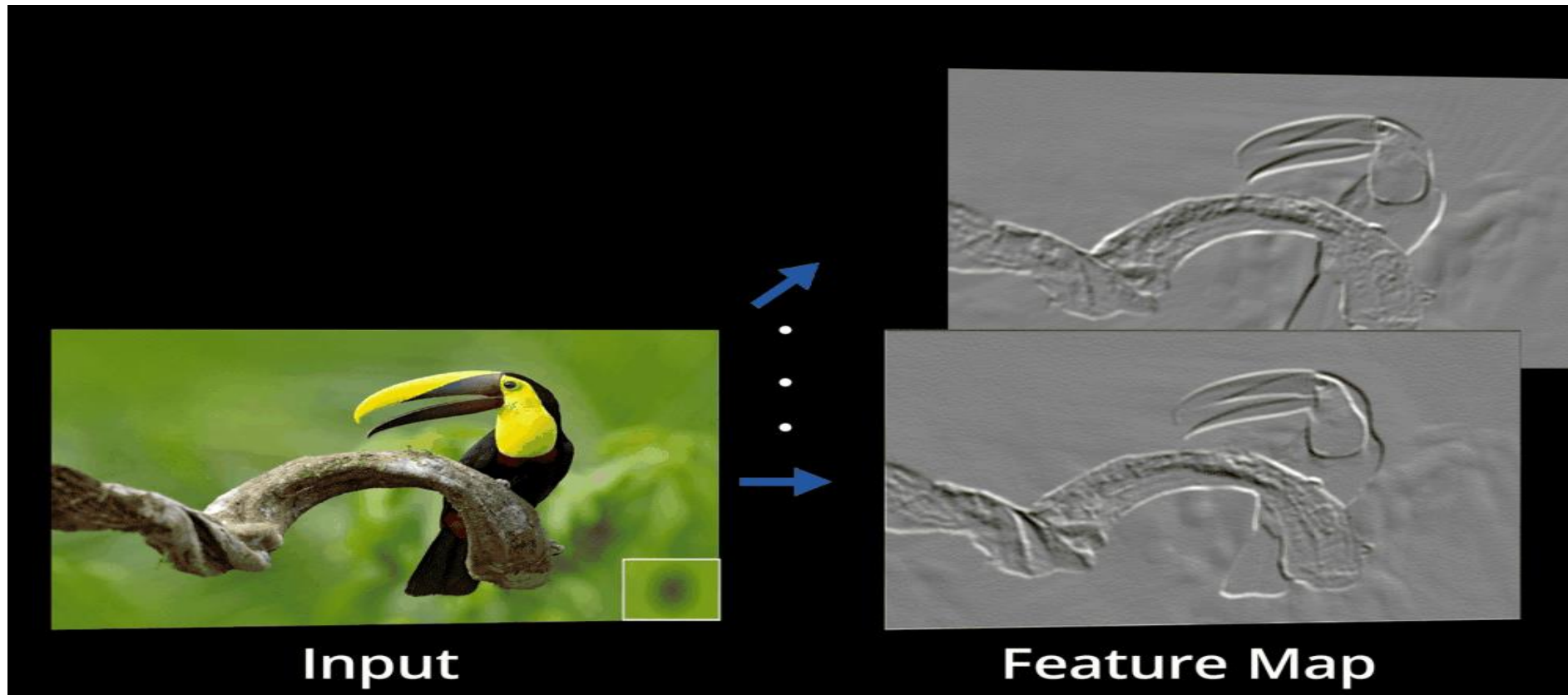
$B_{q+1} = (B_q - F_q + 1)$ along the width of the image

This results in a total of $L_{q+1} \times B_{q+1}$ possible dot products, which defines the size of the next hidden layer.

Thus,
the size of the next hidden layer = $L_{q+1} \times B_{q+1}$



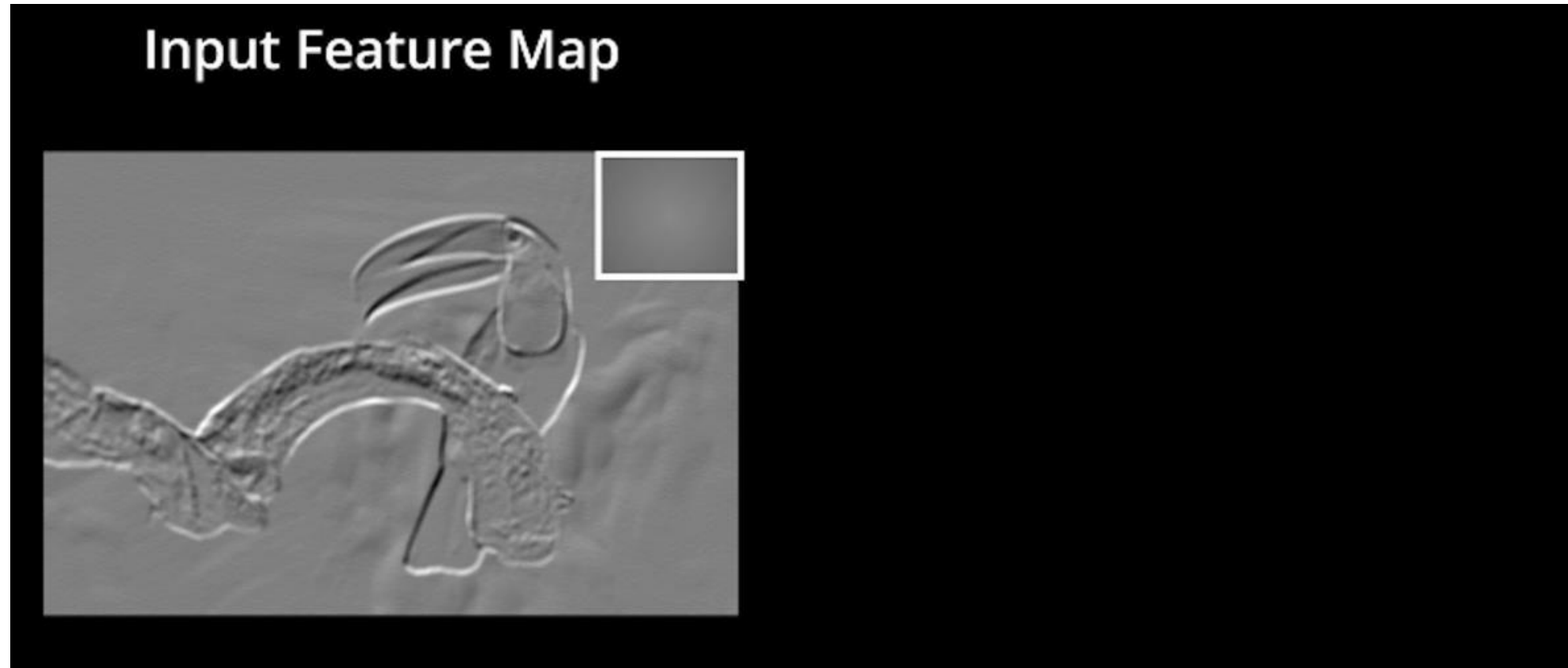
Convolution Layer – output



Sharpen / Rectify feature map

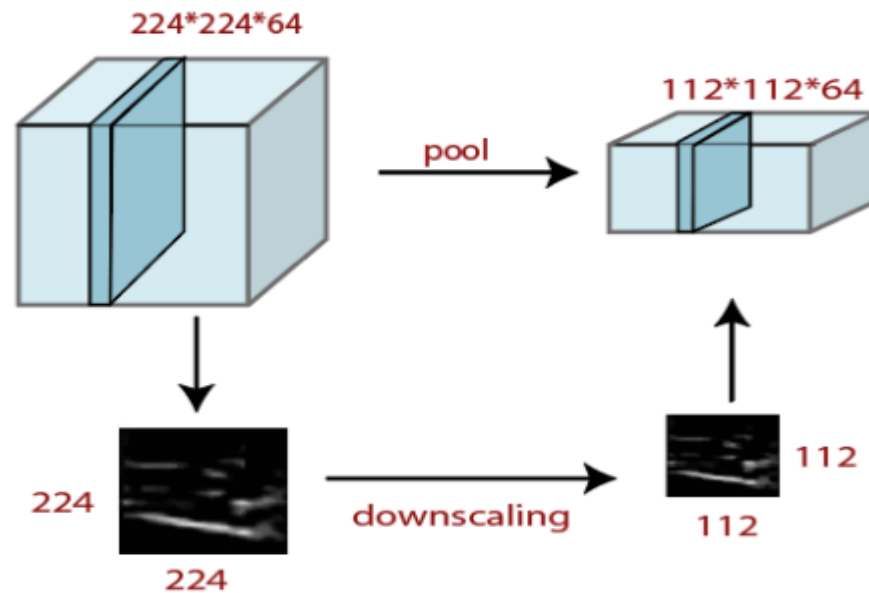
- ❑ Once the feature maps are extracted, the next step is to move them to a ReLU layer.
- ❑ ReLU performs an element-wise operation and sets all the negative pixels to 0.
- ❑ It introduces non-linearity to the network, and the generated output is a rectified feature map.

ReLU - Sharpen / Rectify feature map



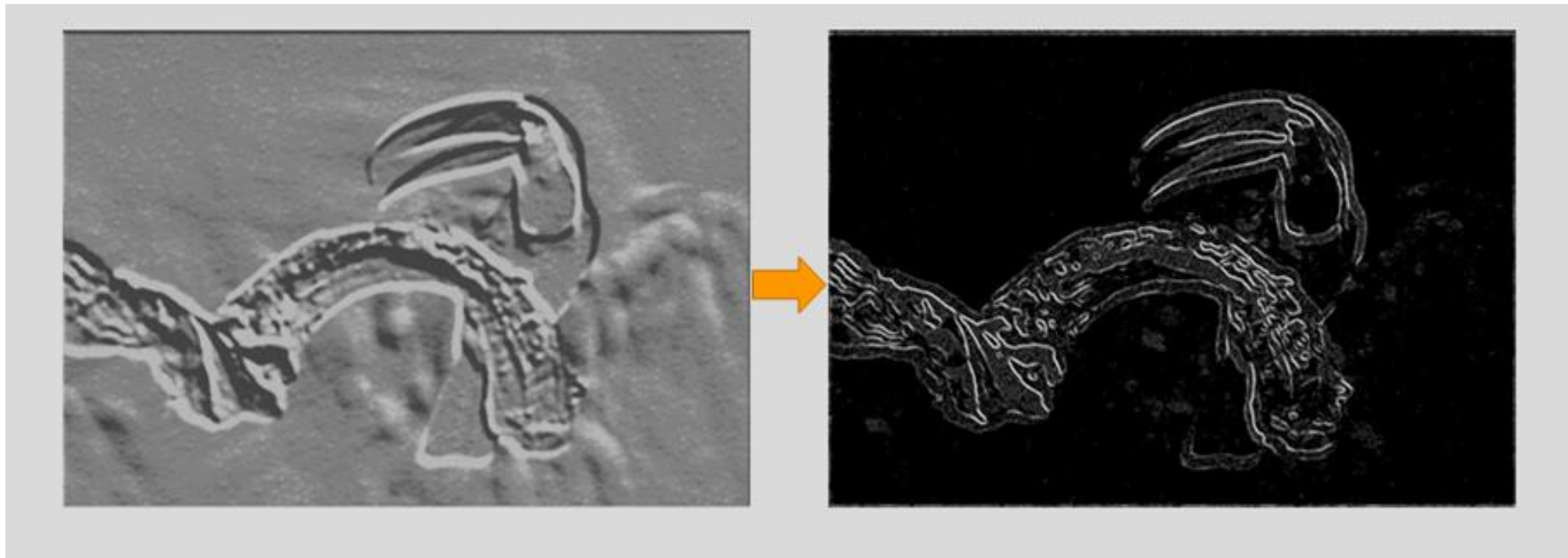
Pooling Layer

- ❑ Pooling is a down-sampling operation that reduces the dimensionality of the feature map.
- ❑ The rectified feature map now goes through a pooling layer to generate a pooled feature map.

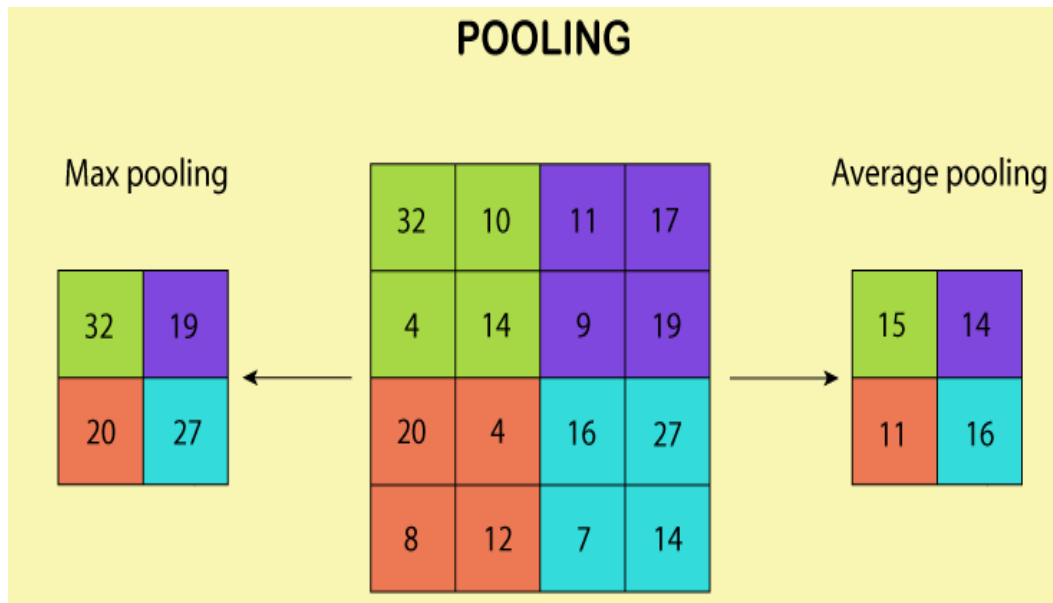


Pooling Layer

- The pooling layer uses various filters to identify different parts of the image like edges, corners, body, feathers, eyes, and beak.



Pooling operation

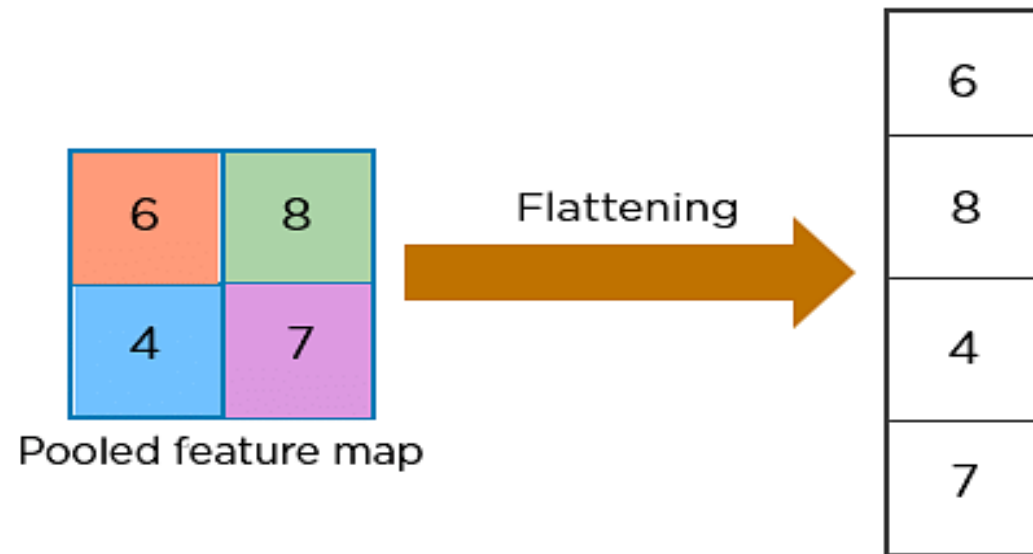


❑ It progressively reduces the spatial size of representation to reduce amount of parameters and computation in network and also controls overfitting.

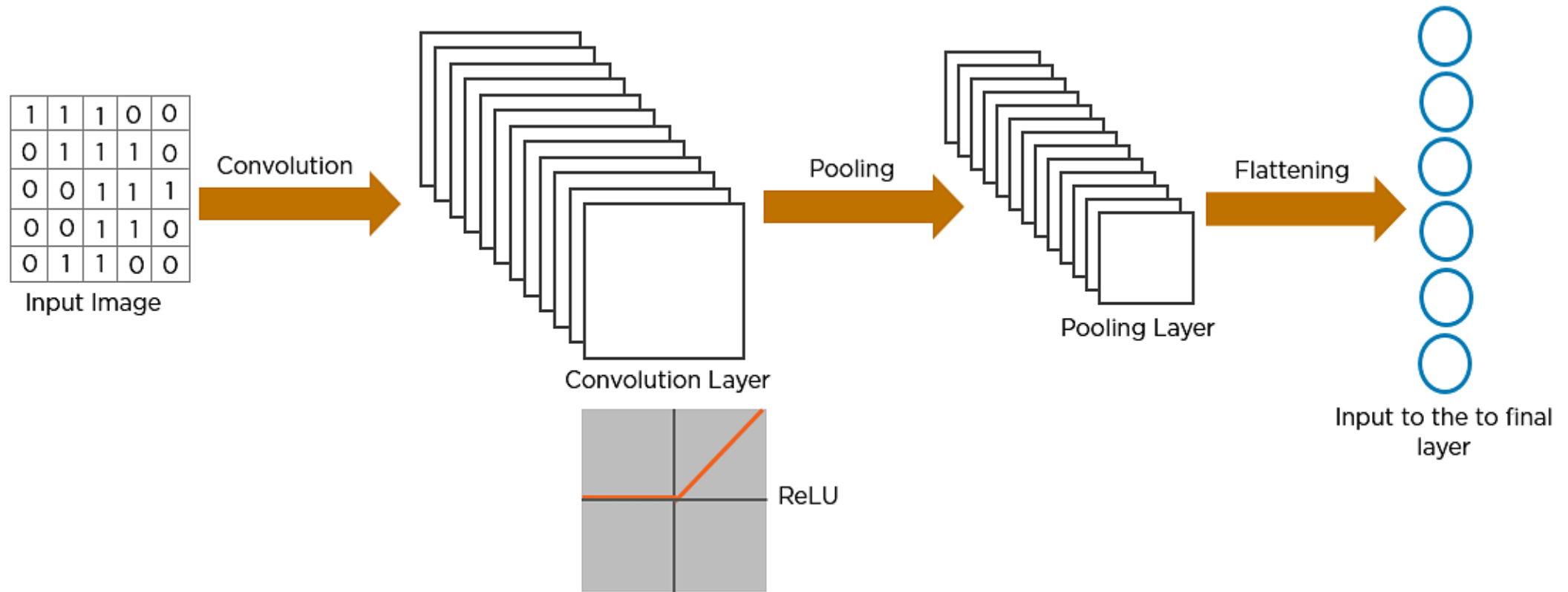
❑ If no pooling, then the output consists of same resolution as input.

Flattening operation

- ❑ The next step in the process is called flattening.
- ❑ Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector.



Flattening operation

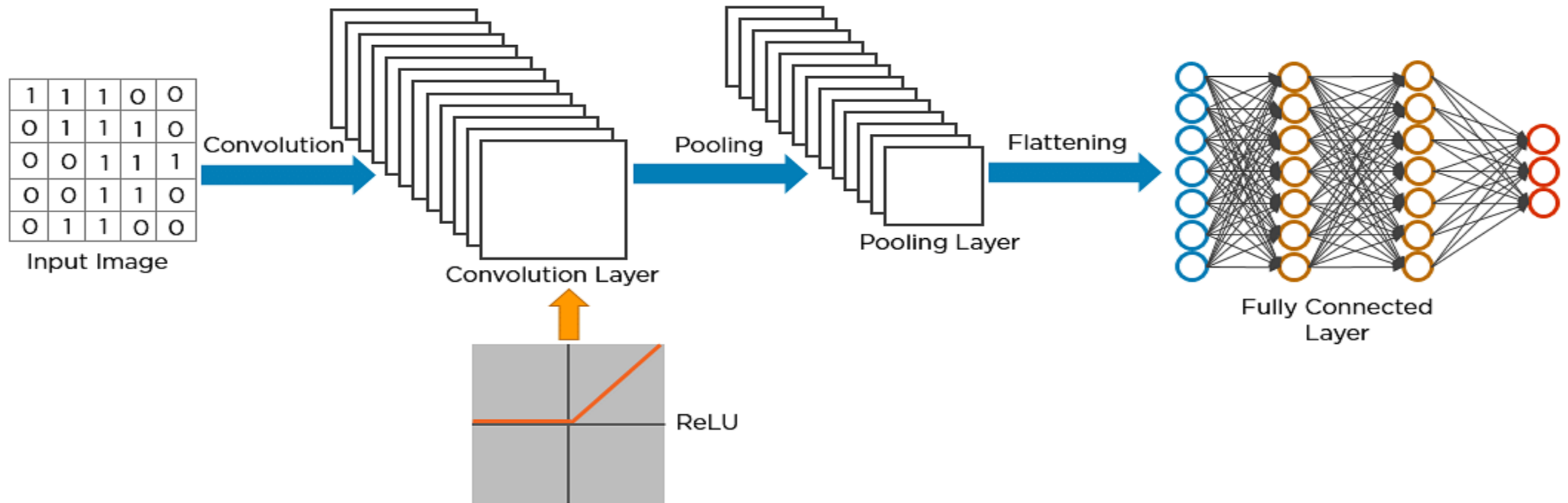


Fully Connected Layer

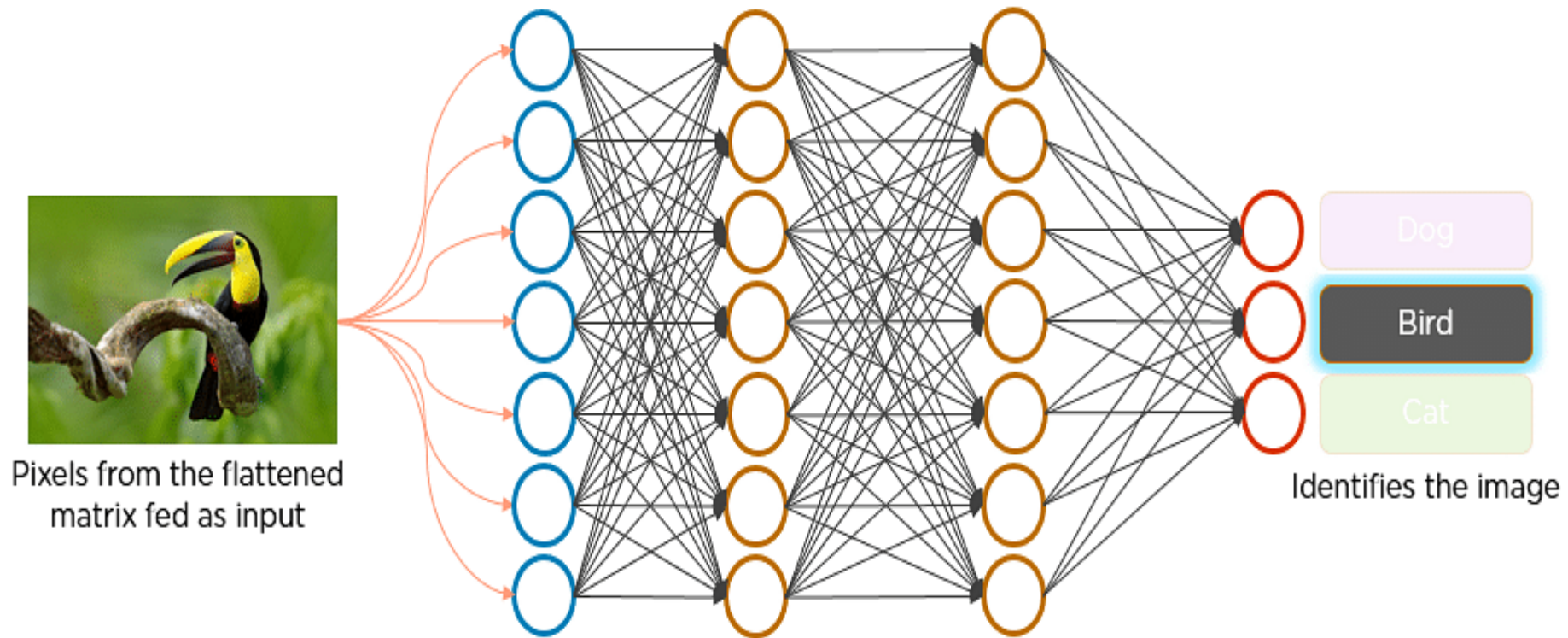
- ❑ The flattened matrix is fed as input to the fully connected layer to classify the image.
- ❑ Fully connected layer looks like a regular neural network connecting all neurons and forms the last few layers in the network.
- ❑ The output from flatten layer is fed to this fully-connected layer.
- ❑ The feature vector from fully connected layer is further used to classify images between different categories after training.
- ❑ All the inputs from this layer are connected to every activation unit of the next layer.

Fully Connected Layer

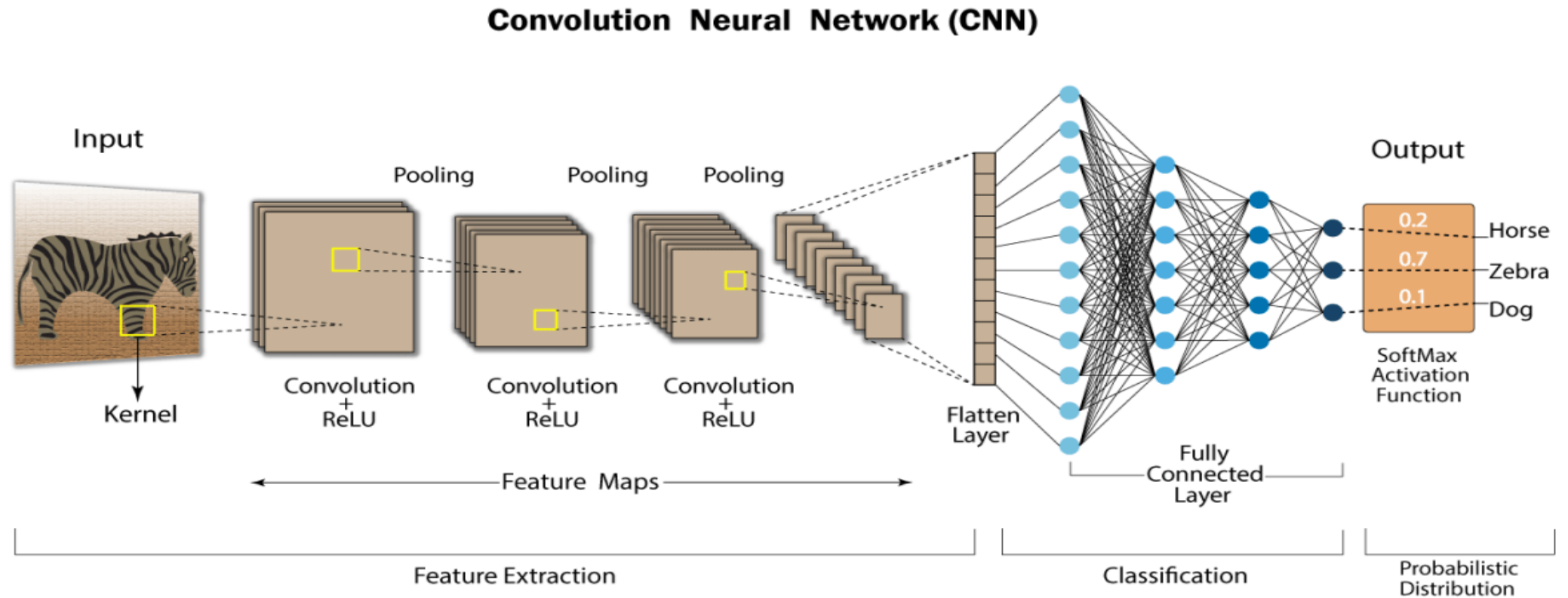
- The flattened matrix is fed as input to the fully connected layer to classify the image.



CNN – classification of image

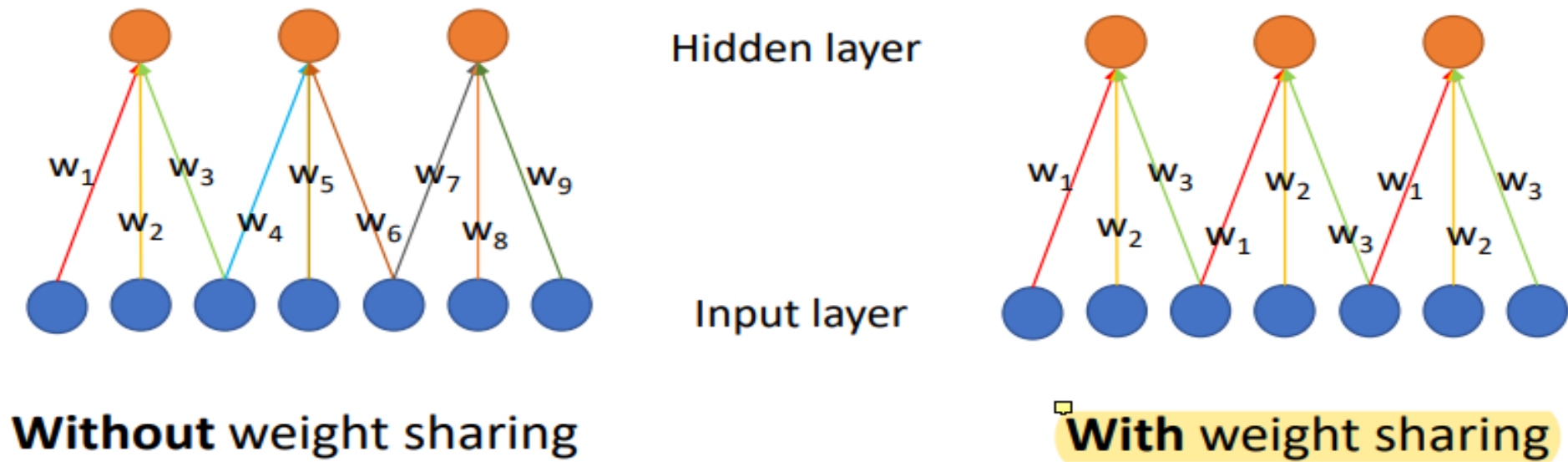


CNN – classification of image

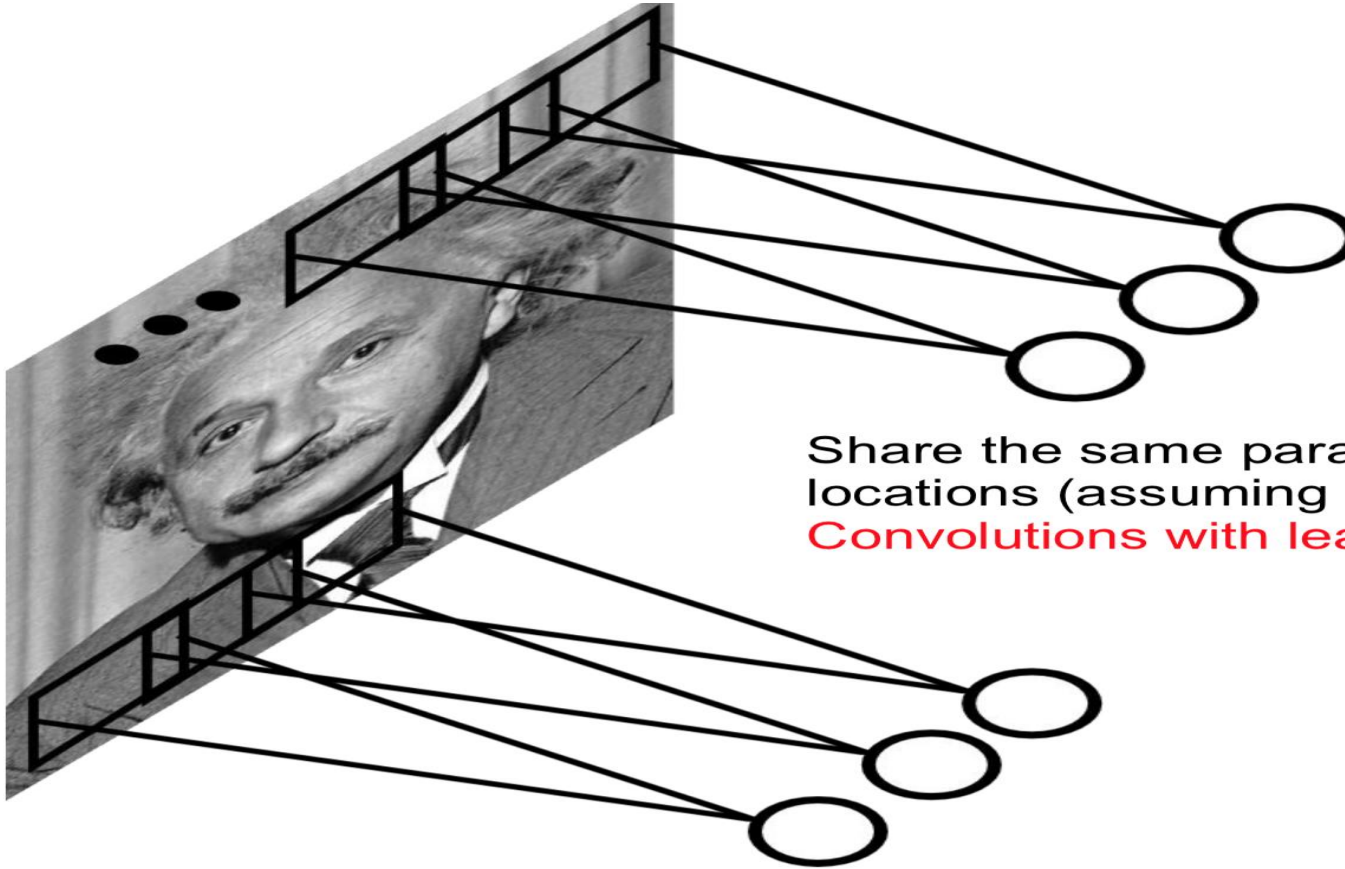


Weight sharing in CNN

- Weight sharing has been a key to the success of convolutional neural networks, as it **forces a neural network to detect common 'local' features across an image by applying the same weights across all input samples (pixels).**



Weight sharing in CNN - Example



Share the same parameters across different locations (assuming input is stationary):
Convolutions with learned kernels

Weight sharing in CNN - Advantages

- ❑ reduces number of weights that must be learned
- ❑ makes feature search insensitive to feature location in the image
- ❑ increases model flexibility

Fully Connected NN and CNN

- Fully connected NN are “**structure agnostic**” no special assumptions needed to be made about the input
- CNN makes **explicit assumptions** that the inputs are images which allows encoding

Variants of Basic Convolution function

Full

- padding
- stride

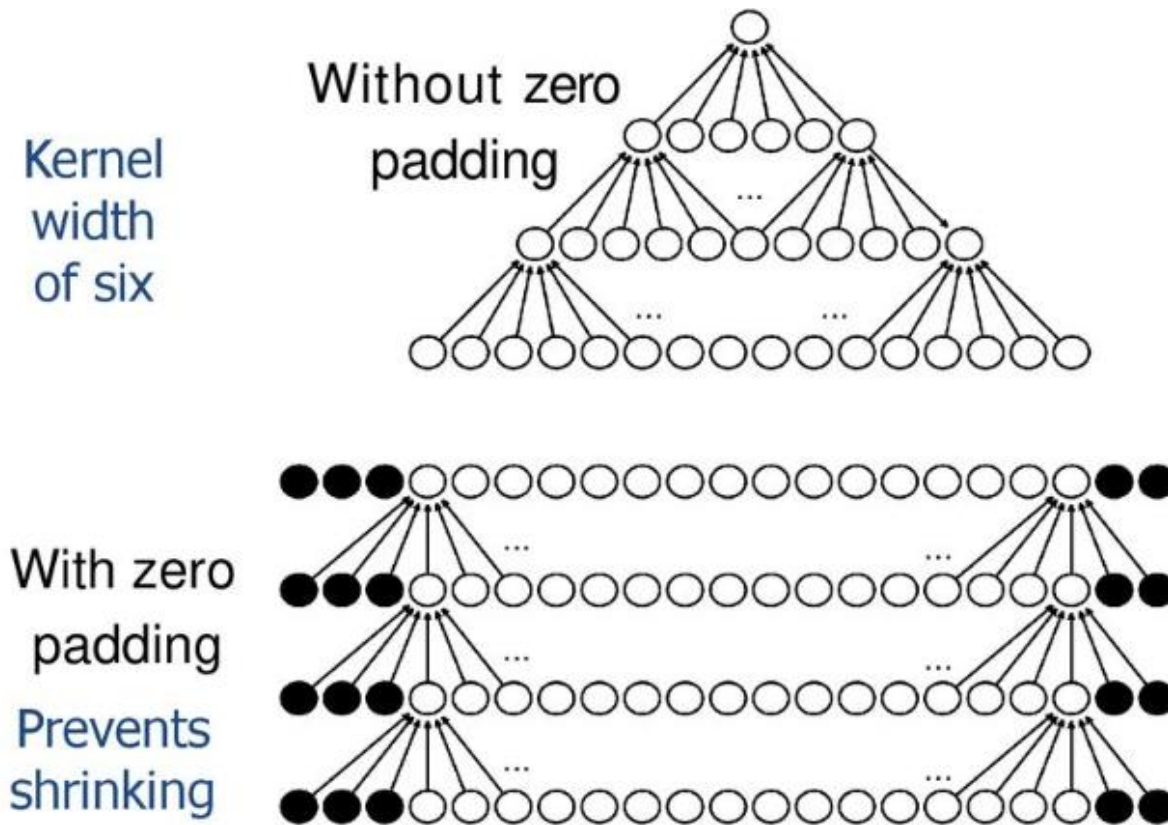
Unshared

- No convolution
- MLP

Tiled

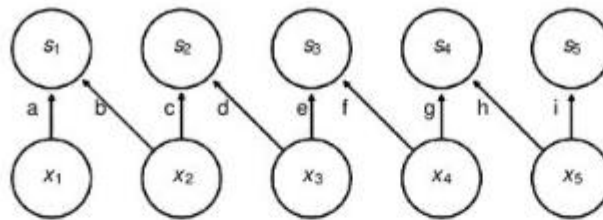
- Neighboring location
- Different filter

Full Convolution

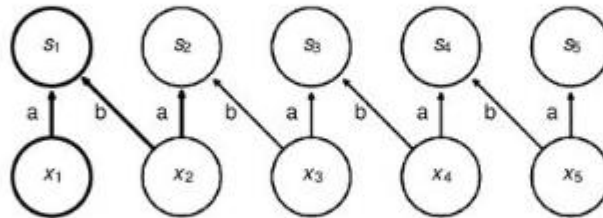


Unshared Convolution

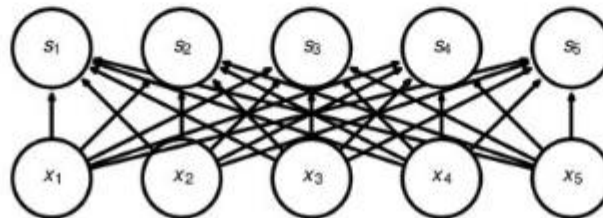
Unshared
convolution



Local connection:
like convolution,
but no sharing

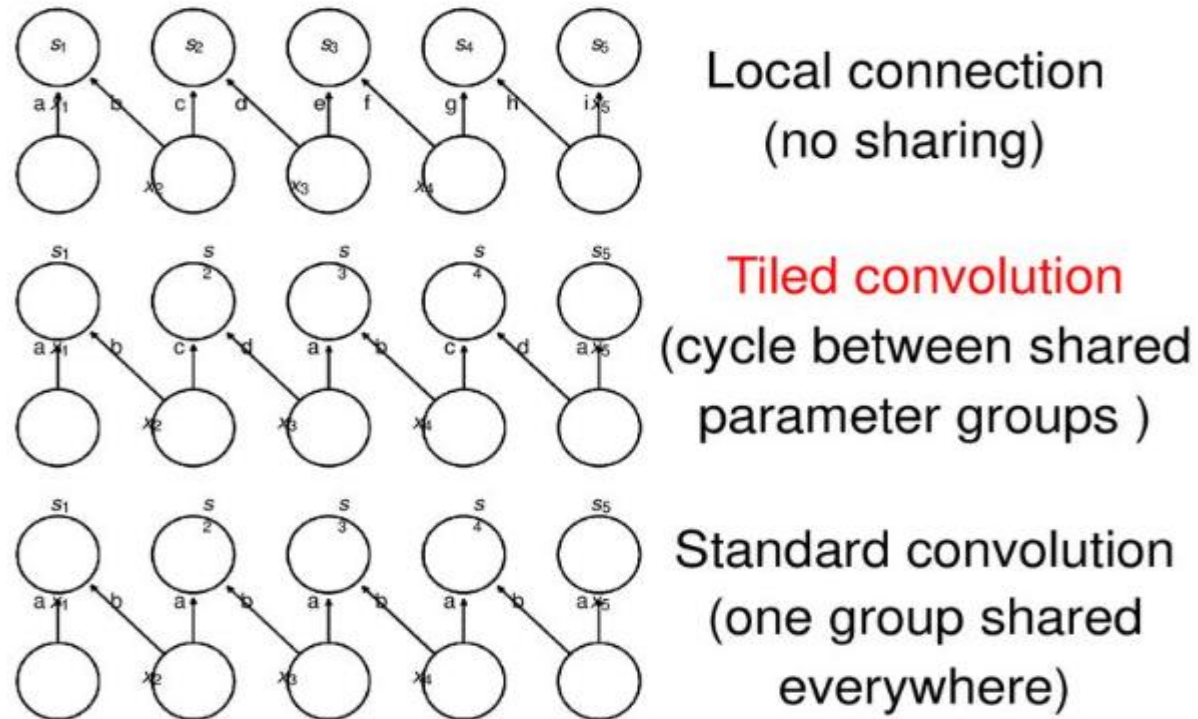


Convolution

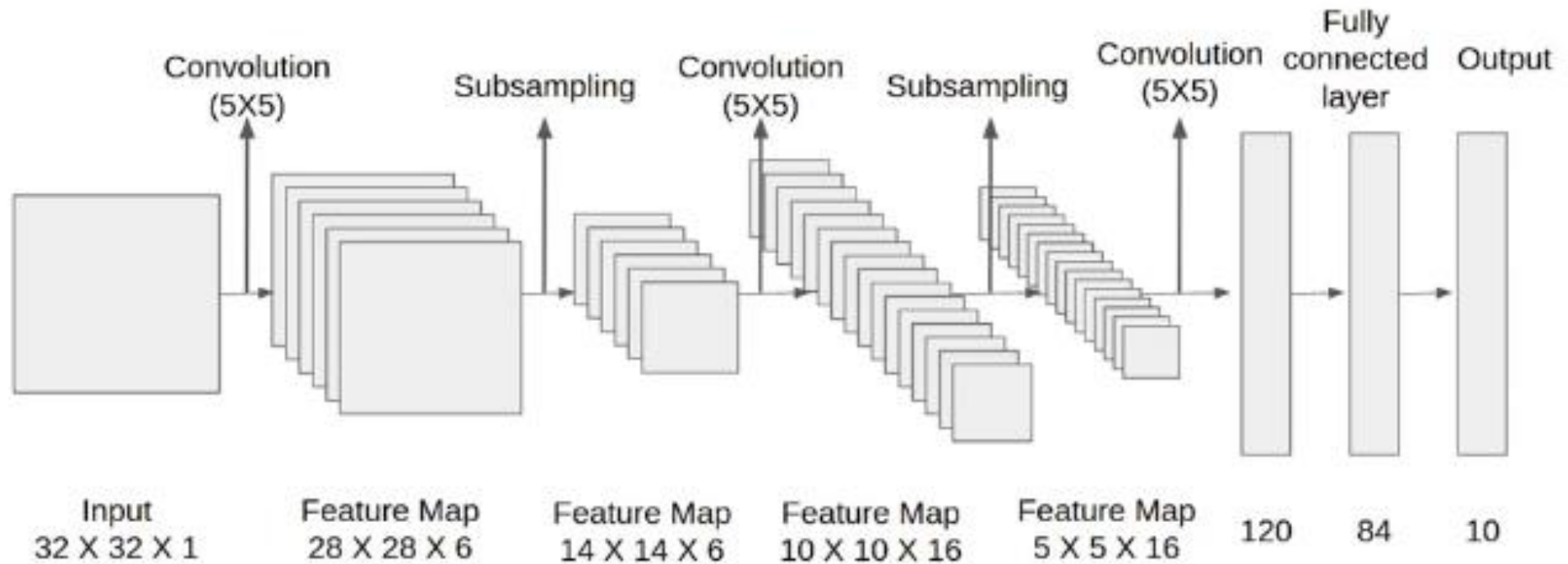


Fully connected

Tiled Convolution



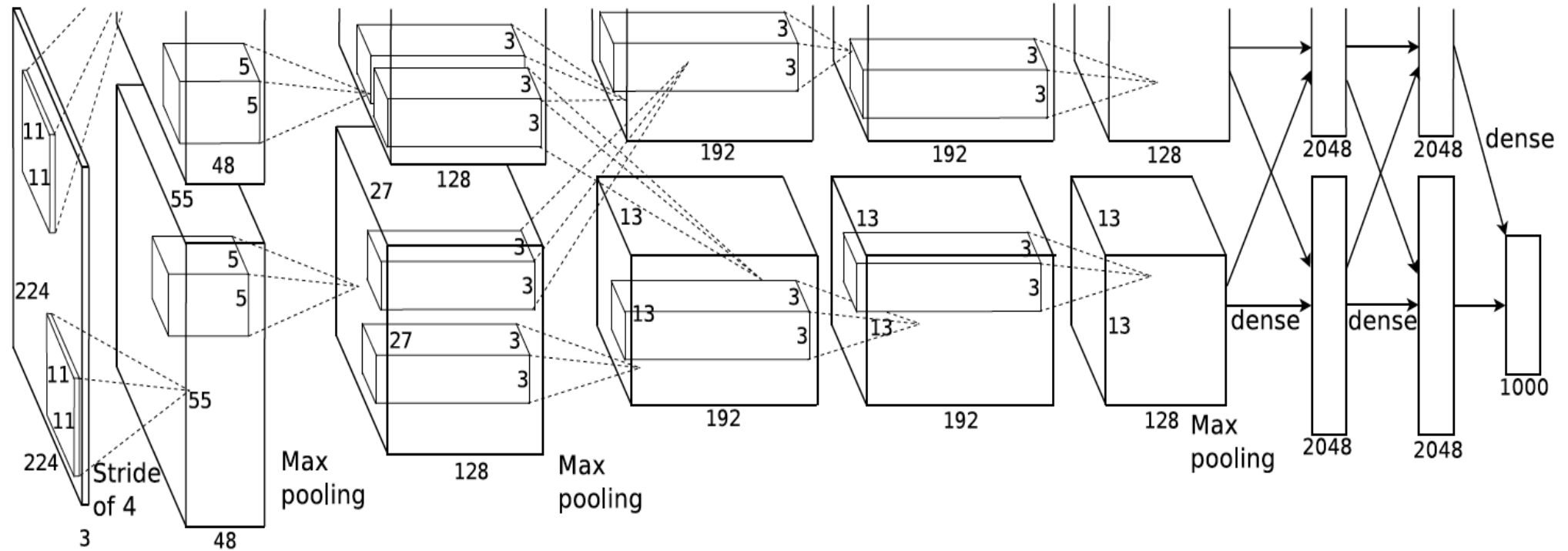
LeNET Architecture – document recognition



AlexNET Architecture – object detection

- AlexNet architecture consists of 5 convolutional layers, 3 max-pooling layers, 2 normalization layers, 2 fully connected layers, and 1 softmax layer.
- Each convolutional layer consists of convolutional filters and a nonlinear activation function ReLU.
- The pooling layers are used to perform max pooling.

AlexNET Architecture – object detection



AlexNET Architecture – object detection

